HACKATHON BUILD REPORT

Platform Walkthrough Agent

An agent that gives the product demo. It reads a product document, plans a walkthrough against the page that is actually on screen, drives the presenter's own signed-in Chrome while narrating, and — when someone interrupts — freezes mid-step, answers from the doc and the live screen, then resumes exactly where it stopped.

Stack	FastAPI · Playwright (CDP) · Claude Opus 5 · Next.js 15 / React 19
Interface	WebSocket event stream + REST setup surface
Browser	Attaches to the presenter's running Chrome — no credentials handled
Tests	25 hermetic (2.7s) + 1 real-browser integration test
Verified	Four full live runs against YouTube and GitHub with a real API key

WHAT THIS DOCUMENT IS

The reasoning, not just the result. It records the three pivots the design went through, why each one was right, and the seven defects that only appeared when the thing was pointed at a real product. The verification section is deliberately specific: every claim in it comes from a run whose output is quoted.

THE THREE CLAIMS

Interruption is the product	The gate is checked between *actions*, not steps. Resume continues the same step — never rewinds, never skips. Asserted by test, not asserted by README.
Nothing about any site is compiled in	The plan is written against an accessibility outline of the page that is on screen. Same build, different document: GitHub, YouTube, or an internal tool.
Real products break agents	Seven defects passed unit tests and failed live — a CSP that killed the cursor, a locator that clicked a logo, a re-plan the loop skipped. Section 6 is the interesting part.

1 · The problem, and the part that is actually hard

Anyone can script a browser. Recording a demo is a solved problem, and so is clicking through one yourself. The engineering that matters is what happens when a customer interrupts — because that is the entire difference between a demo and a screen recording.

The interruption guarantee

When a question arrives mid-step:

- **The executor clears an** `asyncio.Event`**.** The loop parks at `await self._gate.wait()` — no polling, no busy-wait — and the gate is checked between *actions*, not just between steps, so the browser stops where it stands.
- **Claude answers** with the product document as cached context and the *current screenshot* attached, so the answer is grounded in what the customer is looking at rather than in what the plan said would be there.
- **The Event is set.** Execution continues at the same `current_index` — the pointer only advances after a step *fully* completes, so resume never rewinds and never skips.

That guarantee is a test, not a claim. `test_pause_freezes_and_resume_continues_same_step` asserts that across a pause no action is replayed and none is dropped.

Pause is not instantaneous, on purpose

A Playwright action already in flight runs to completion before the loop parks. Cancelling mid-action would leave a half-typed field on screen in front of the customer. Measured worst case is ~1.5s, and the UI is honest about it: the badge reads **PAUSING…** while the browser is still settling and only flips to **PAUSED** once it has actually stopped. The status the client requests and the status the executor reports are deliberately different events.

2 · The thinking: three pivots

The design changed shape twice after the first working version. Both changes were driven by the same question — *what does this have to know in advance?* — and the answer kept being *less than it currently does*.

Pivot 1 — from a scripted GitHub demo to a site-agnostic agent

The first build knew about GitHub. It had a `TARGET_URL`, a `DEMO_REPO`, and a session cookie pasted into the environment. It worked, and it was the wrong shape: every new product meant a code change, and the cookie meant the repo held a live credential.

The replacement inverts the flow. The presenter uploads the product document at run time, says what to demo in plain English, and names one of their open browser tabs. The agent attaches to that tab, reads an accessibility outline of the page, and plans against *those* controls. Nothing about any site is compiled in — the same binary demos GitHub, YouTube, or an internal tool.

The credential problem dissolved rather than being solved. Attaching to a browser the presenter is already signed into means there is no login code path to write, no cookie to store, and nothing for the agent to type.

Pivot 2 — the browser is the presenter's, not ours

Launching a clean automation window is easier and worse. It is signed out, it looks nothing like the product the audience uses, and the sterile "Chrome is being controlled by automated software" frame is the first thing on screen. Attaching over the DevTools protocol to a real, logged-in Chrome fixes all three at once — the

audience watches the actual product, in the actual browser, with the presenter's actual account.

Pivot 3 — locking it down for the hackathon

For a judged run, choice is a liability: every input is something to fat-finger on stage. `DEMO_LOCKED=true` reduces the UI to one card and one button, and the *server* discards whatever the client posts and substitutes the preset — a stale browser tab cannot run something else.

This was implemented as a preset over the general engine rather than by deleting the general path. `DEMO_LOCKED=false` restores the site-agnostic agent with no other change, so the answer to "does this only do YouTube?" is a live demonstration rather than an assurance.

3 · Architecture

```
Next.js UI  ==WebSocket==>  FastAPI  ===>  StepExecutor  ===>  Playwright
    |          + REST          |              |                    | CDP
setup panel                session state   asyncio.Event            v
chat + Q&A                                 pause/resume     your own Chrome
live screenshot                 |                          (already signed in)
step progress              Claude (Opus 5)
                  plan / narrate / answer / re-plan
```

*The ordering is the design: **attach** → **pick the tab** → **read that page** → **plan**. Planning happens only after the agent can see the live DOM, which is what lets one prompt work on a site the code has never seen.*

Modules

File	Responsibility
agent/browser.py	CDP attach, auto-launch, tab selection, semantic locators, navigation scope enforcement
agent/tabs.py	Turns “the youtube one” into one specific open tab — or refuses. Playwright-free so it is unit-testable in milliseconds
agent/docs.py	Uploaded md / txt / html / PDF / docx → text, content-addressed on disk
agent/doc_parser.py	Document + live page outline → executable StepPlan via structured outputs
agent/narrator.py	Pre-generates narration; answers live questions; classifies question vs. redirect
agent/step_executor.py	The pause/resume loop, question breaks, mid-step re-planning
agent/session.py	Per-viewer orchestration: preset, sign-in gate, planning, interruptions
agent/overlay.py	On-page cursor, spotlight and caption drawn into the DOM
agent/memory.py	Workflow memory — recall a plan instead of re-planning

Why the overlay is drawn into the page

The OS mouse pointer is *not* composited into screenshots or into most screen-share capture paths. Without an overlay the audience watches buttons activate by themselves. So the agent injects its own cursor, glides it to each target, pulses on click, spotlights the element, and burns the current narration into the page as a caption bar — which also means the browser window is self-explanatory when it is shared *without* the app's chat panel.

The action vocabulary is closed

Claude chooses from six verbs — navigate, click, fill, press, wait, highlight — and addresses elements by ARIA role plus accessible name. It never emits a CSS selector or an XPath. Constraining the vocabulary is what

makes an LLM-authored plan safely executable; products rewrite their class names constantly, but what a control *is* and what it is *called* survive a deploy.

4 · Decisions worth defending

Narration is pre-generated; answers are live

Every narration line comes back in one Claude call at plan time. Calling Claude between steps would make every transition visibly stall on a round trip. Only Q&A; — where the customer is already waiting — happens live.

Answering and re-planning share one API call

“Can one issue live on two boards?” is a question. “Skip to the board” is a redirect. Deciding which is the same judgement as answering, so `QAResponse` returns `answer`, `wants_plan_change` and `requested_focus` together. A second classifier call would double the pause the customer sits through.

Scope is derived at run time and enforced in code

The navigation allowlist is the chosen tab's host plus its registrable domain — nothing broader. It is checked twice: once when sanitising the plan, and again immediately before `page.goto`. With no tab chosen the allowlist is empty and *everything* fails closed. The prompt asks; the code enforces. An LLM-authored plan is driving a browser signed into real accounts, so that check does not get to live in a prompt.

It refuses to guess which tab

Tab matching scores host equality, domain suffix, and title/path tokens, and returns nothing below a threshold — the UI then shows the tab list. Opening the wrong customer's tab on a screen share is far worse than one extra question. The one place guessing *is* allowed is a freshly launched empty browser, where there are no tabs to confuse and the alternative is a blank window.

It stops and asks

After every step the agent asks the room for questions and holds; at the end it holds a full minute. The window *restarts* whenever someone actually asks, because one question almost always has a follow-up. A question asked in the closing window can still extend the demo — the agent re-plans and performs the new thing rather than ending on an answer nobody saw.

There is always a plan

No API key, no network, a 500 from Claude — the agent falls back to a plan built from the document's own headings, with wait-only steps. It never invents navigation it cannot justify. A live demo must never open with a stack trace.

Length comes from the document

A document that says “about two minutes, plan exactly three steps” overrides the planner's default of four to six. The presenter who wrote down how long they have has already made that decision.

5 · Safety model

Concern	How it is handled
Credentials	Never typed. Authentication comes from the attached browser profile; there is no login-form code path and no stored cookie. The only secret in the repo is ANTHROPIC_API_KEY.
Navigation	Allowlist derived from the selected tab, enforced twice in code, fails closed when empty.

Destructive actions	The planner is instructed to prefer showing over changing, and never to delete, send, publish, pay, or alter account settings.
Signed-in state	Detected before planning. A signed-out browser gets a plan that points at where account features live instead of clicking things it cannot complete.
The presenter's browser	An attached browser is never closed on teardown — stop() disconnects. Auto-launch only ever fires for a CDP URL on this machine.

6 · Seven defects that only a real product surfaced

Every one of these passed unit tests and failed in front of a live page. They are the strongest argument in this report for testing an agent against the actual thing it will be pointed at.

01 · Trusted Types killed the entire overlay

Symptom	On YouTube the cursor and caption never appeared. Spotlights worked, so it looked cosmetic and random.
Root cause	YouTube enforces a Trusted Types CSP. The cursor was built with `innerHTML`, which throws under that policy — taking the whole build function down with it, including the caption. Spotlights survived only because they are created per call.
Fix	Build the SVG with `createElementNS` and wrap the builder in a try/catch, so a hostile page degrades the overlay instead of killing a step.

02 · Single-page apps silently dropped the overlay

Symptom	The cursor and captions vanished part-way through a demo and never returned.
Root cause	SPAs re-render `document.body` and take injected nodes with them. The builder trusted a boolean and refused to rebuild.
Fix	Check `isConnected` instead, and restore the caption text and cursor position on rebuild so a mid-narration navigation is invisible.

03 · Substring name matching clicked the wrong element

Symptom	Every click meant for the sidebar “You” entry landed on the YouTube logo.
Root cause	Locators used substring matching, so `name="You"` also matched “YouTube Home”, which sits earlier in the DOM. A union locator returns matches in *DOM order*, so the page chose, not us.
Fix	Try exact names before substrings, in explicit passes, and only consider visible matches — apps keep duplicate markup for responsive layouts.

04 · A re-plan mid-step was skipped entirely

Symptom	“Show me how to filter the data” produced a tidy answer and no change on screen — the demo carried on as though nobody had spoken.
Root cause	The run loop holds the executing step in a local variable. Swapping the plan mid-step finished the *old* step and then advanced past the new one.
Fix	A flag set by `replace_plan` breaks the action loop and re-enters *without advancing*, so the step now at that index runs. Pinned by a test that fails against the old code.

05 · The sign-in prompt deadlocked the WebSocket

Symptom	The connection died on its own keepalive the first time the agent asked a question with buttons.
Root cause	`start()` awaited the presenter's reply inline on the receive loop — the same loop that had to read that reply.
Fix	Run it as a background task, with failures reported to the client rather than swallowed by the event loop.

06 · The search step typed into a button

Symptom	The search never ran. The trace still said `typed into 'Search'`, and the failure surfaced one step later as a missing filter control.
Root cause	YouTube's search field is `role="combobox"`, not `textbox`. The role fallback chain reached the *Search button*; typing into a button raises nothing and does nothing.
Fix	A fill now only ever considers editable roles, and verifies the resolved element is editable before typing — otherwise it reports the failure instead of claiming success.

07 · Chrome 136+ and 151+ both refuse the debug port

Symptom	`connect_over_cdp` failed against the presenter's stable Chrome with “Browser context management is not supported”.
Root cause	Chrome 136 refuses the debug port on the default profile; Chrome 151 rejects the CDP handshake without `--enable-automation`. Determined empirically by testing flags one at a time.
Fix	All three flags are in the launch command the agent runs itself, and the app surfaces that exact command wherever attaching can fail.

7 · Verification

Four full runs with a real API key against live products, plus the hermetic suite. Everything below is quoted from run output.

Same build, two products, two documents

```
doc: "Watch Later - product flow"     tab: "youtube"   -> www.youtube.com
  6 steps planned from the live page  6/6 executed, 0 failed actions
  interrupt after step 2 -> pausing -> paused -> answered -> resumed at step 3
  the answer: cross-device sync isn't in the doc, so it declined to guess

doc: "Explore - product flow"         tab: "github"    -> github.com/explore
  6 steps planned from the live page  in-scope hop to github.com/trending
  interrupt: "Can I filter Explore by language?" -> answered from the page
```

The only difference between the two runs is the uploaded document, the sentence describing the flow, and the tab named.

An interruption redirecting the browser, not just the answer

```
>>> "show me how to filter the results"
 33.0s [answer]  see the row of chips under the search box...
 33.0s [status]  Re-planning around: filtering with the category chips
 65.5s [step]    clicked 'Jet engines'   -> results reshape
 88.4s [step]    typed a query, pressed Enter, results landed
156.6s [complete]                        steps: [1,2,3] | errors: []
```

The locked demo, ignoring hostile input

```
client posted:  doc_id="bogus"  focus="something else entirely"  tab="salesforce"
  0.9s  Demoing in: YouTube          <- the preset, not what the client sent
  0.9s  [PROMPT] You're not signed in. Do you want to sign in first...?
        -> "No, continue signed out"
 18.2s  PLAN: 3. Open the sidebar and point at You :: click Guide,
                 highlight You, highlight 'Sign in'
 55.6s  [complete]
```

That last step is the signed-out constraint working: told the browser has no account, the planner pointed at where the saved queue lives and at the sign-in control, rather than planning a click it could not complete.

Attach, targeting, and scope

```
attached to 3 tabs
match("the iana tab") -> www.iana.org      match("salesforce") -> None (refused)
selected www.iana.org  scope: ['iana.org', 'www.iana.org']
page_outline           31 controls read from a site the code has never seen
navigation to https://evil.example/ -> "skipped - outside this demo's scope"
Chrome still running after stop(): True    (attach disconnects, never closes)
```

Pacing, measured

Measurement	Result
Cold start: no browser → attached, on the product	2.7 s
Pause request → browser truly parked	1.52 s (in-flight action finishing)
Actions started after parked	0
Live frame stream	3.4 fps sustained, ~39 KB per frame
Two-minute demo, three steps with question breaks	~87 s to complete
Hermetic test suite	25 passed in 2.7 s

What is not proven

Two things, stated plainly. The *yes, I'll sign in* branch of the sign-in gate has unit coverage for its prompt mechanics but has not been exercised through a real interactive login — the agent deliberately has no way to do that itself. And every live run so far has been driven by a scripted WebSocket client rather than a human clicking the UI; the UI has been verified by screenshot and type-check, not by a full click-through.

8 · Harness integration

File	What it does
walkthrough-agent-skill.yaml	Worker Agent Skill: trigger phrases, four-phase instructions (targeting, planning, execution, interruption), RBAC, audit capture, derived domain allowlist, denied actions
pipeline.yaml	Build & Test → Register Skill → Deploy, with StageRollback on failure
rollback-policy.yaml	Rollback triggers, session version pinning, approval gates

Registering the walkthrough as an Agent Skill is what turns a demo script into shared infrastructure — versioned, governed, and callable from the catalog by any team. Sessions pin to the skill version they started on, so a deploy can never change the demo out from under a customer who is mid-call.

The pipeline's real gate is the pause/resume suite. A walkthrough that rewinds when someone asks a question is worse than no demo at all, so that behaviour is verified on every commit rather than rehearsed before every call. The rollback policy watches for the failure that actually hurts — `stuck-pause`, a session that paused for a question and never resumed. That is a browser frozen mid-sentence in front of a customer.

9 · Running it

```
cd backend && python3 -m venv .venv \
  && .venv/bin/pip install -r requirements.txt \
  && .venv/bin/playwright install chromium
cp backend/.env.example backend/.env      # fill in ANTHROPIC_API_KEY
cd backend && .venv/bin/uvicorn main:app --reload --port 8000
cd frontend && npm install && npm run dev
```

Starting a walkthrough opens a debug-enabled Chrome on a dedicated profile if one is not already running — roughly three seconds, on the product named. Sign into that window once; the profile persists. Health check before a call: `claude_configured` and `chrome_reachable` must both be true, and `chrome_command` is the exact command to fix the second one by hand.

Bonus features delivered: voice narration (Web Speech API, cancels the instant someone interrupts) · plan-changing interruptions (completed steps preserved verbatim, never replayed) · workflow memory (successful plans persist; the second demo of the same product skips planning entirely).